hashlock.

Security Audit

Spiral Stake (DeFi)

September 2025

info@hashlock.com.au

Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Spiral Stake team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

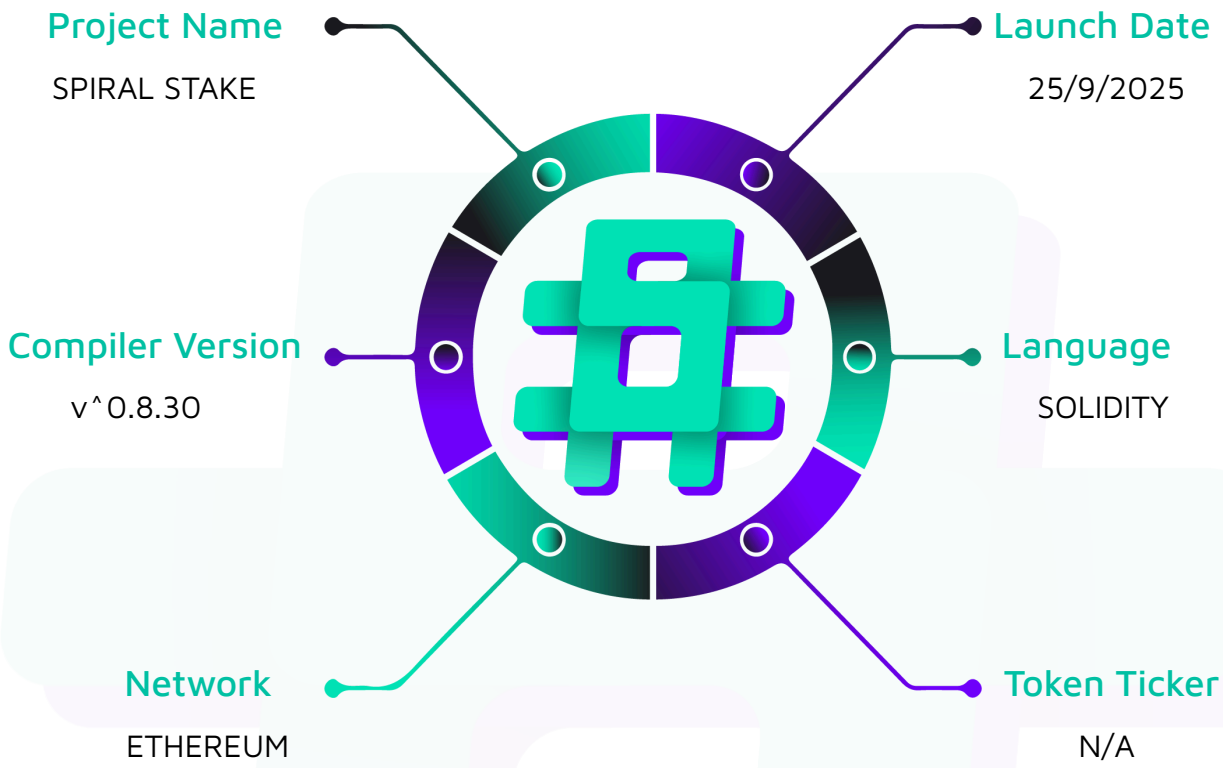
Project Context

Spiral Stake is a non-custodial leveraged looping protocol designed exclusively for maximizing stablecoin yields. By locking in fixed yields from Principal Tokens (PTs) of staked stablecoins and flash leveraging them at conservative loan-to-value ratios, the protocol delivers amplified returns without compromising security.

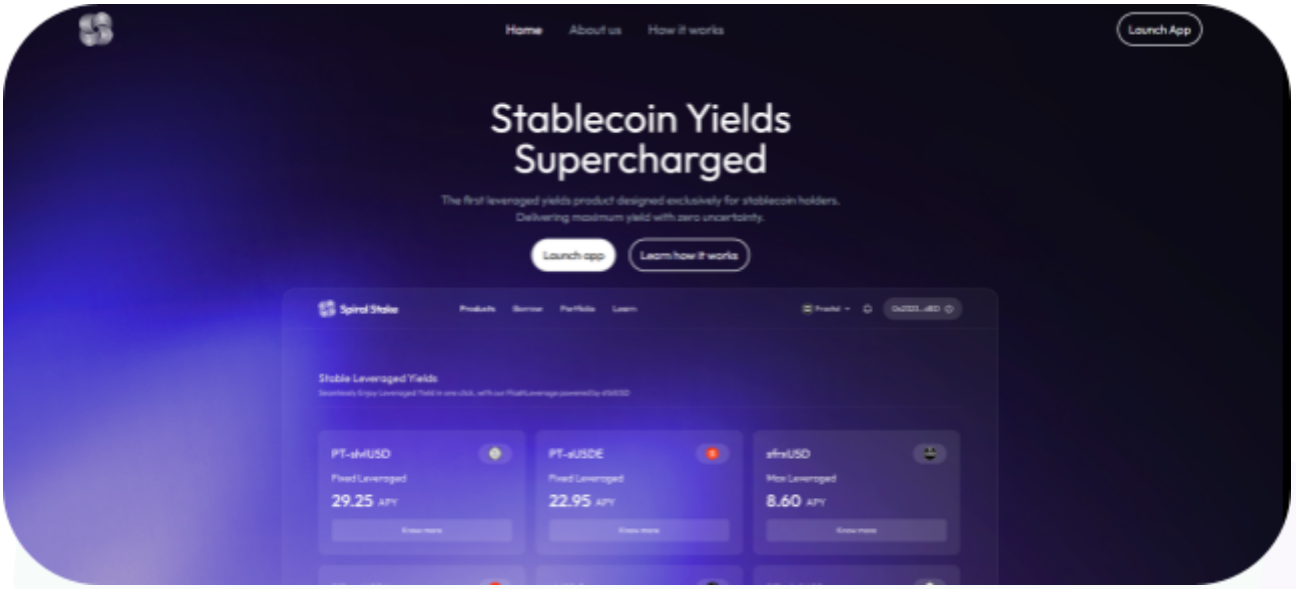
Building on Pendle's PT infrastructure, which established the fixed yield foundation - Spiral Stake optimizes borrowing costs across Morpho lending pools to enable stable, predictable leveraged yields that function like high-yield bonds with DeFi upside potential.

Project Name: Spiral Stake
Project Type: Defi
Compiler Version: ^0.8.30
Website: https://www.spiralstake.xyz/
Logo:

Visualised Context:



Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the Spiral Stake project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	**Spiral Stake Smart Contracts**
Platform	**Ethereum / Solidity**
Audit Date	**September, 2025**
Contract 1	FlashLeverage.sol
Contract 2	FlashLeverageCore.sol
Contract 3	MarketPositionManager.sol
Contract 4	SwapAggregator.sol
Contract 5	UserProxy.sol
Audited GitHub Commit Hash	f6613010a3876334abd95d3df5985ed9a1bc8907
Fix Review GitHub Commit Hash	3b1fe5c1c13dafbfc71db76702c0c00c9a9742da

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved or acknowledged.

Hashlock found:

1 High severity vulnerability

3 Medium severity vulnerabilities

7 Low severity vulnerabilities

1 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
FlashLeverage.sol - Allows users to: - Supply collateral and flash leverage their position - Unleverage their position collateral - Allows admins to: - Add collateral support - Update treasury address	**Contract achieves this functionality.**
FlashLeverageCore.sol - Further executes leverage and unleverage operations - Keep separate tracking of users' positions - Swap collateral token to loan token and vice versa - Clones a minimal user proxy for Morpho execution - Allows admin to : - Admins can add a Manager contract	**Contract achieves this functionality.**
Userproxy.sol - Execute morpho operations via FlashLeverageCore	**Contract achieves this functionality.**

Code Quality

This audit scope involves the smart contracts of the Spiral Stake project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Spiral Stake project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] FlashLeverage#unleverage - Attacker can DoS User Leveraged Position

Description

Because `FlashLeverage::unleverage` uses the `LeveragePosition` struct to retrieve funds from the Morpho market, the attacker can maliciously withdraw an extremely small part of shares to make the user not able to withdraw assets from the Morpho market.

Vulnerability Details

```
function _unleverage(
      address user,
      uint256 positionId,
      LeveragePosition storage position,
      address pendleSwap,
      address tokenRedeemSy,
      SwapData calldata swapData,
      LimitOrderData calldata limitOrderData
  ) internal returns (uint256 amountReturned) {
      // Execute core unleverage operation
      i_flashLeverageCore.unleverage(
          user,
          UnleverageParams({
              collateralToken: position.collateralToken,
              loanToken: position.loanToken,
              desiredLtv: position.desiredLtv,
```

```
>>>             sharesToBurn: position.sharesBorrowed,
                amountCollateralToWithdraw: position.amountLeveragedCollateral,
                pendleSwap: pendleSwap,
                tokenRedeemSy: tokenRedeemSy,
                swapData: swapData,
                limitOrderData: limitOrderData
            })
        );
        ....
    }
```

As you can see, the above function, which is called by `FlashLeverage::unleverage`, which eventually calls `FlashLeverageCore::_repayAndWithdrawCollateralViaProxy`, where shares are repaid and collateral withdrawn. But if you look into the `Morpho::repay` function, it tries to withdraw the shares that the user inputs, i.e., the function `FlashLeverageCore::_repayAndWithdrawCollateralViaProxy` inputs.

```
  function repay(
        MarketParams memory marketParams,
        uint256 assets,
>>>     uint256 shares,
        address onBehalf,
        bytes calldata data
    ) external returns (uint256, uint256) {
        Id id = marketParams.id();
        require(market[id].lastUpdate != 0, ErrorsLib.MARKET_NOT_CREATED);
        require(UtilsLib.exactlyOneZero(assets, shares), ErrorsLib.INCONSISTENT_INPUT);
        require(onBehalf != address(0), ErrorsLib.ZERO_ADDRESS);
        _accrueInterest(marketParams, id);
```

```
            if (assets > 0) shares = assets.toSharesDown(market[id].totalBorrowAssets,
market[id].totalBorrowShares);

                        else   assets   =   shares.toAssetsUp(market[id].totalBorrowAssets,
market[id].totalBorrowShares);

>>>     position[id][onBehalf].borrowShares -= shares.toUint128();

        market[id].totalBorrowShares -= shares.toUint128();

                                                    market[id].totalBorrowAssets      =
UtilsLib.zeroFloorSub(market[id].totalBorrowAssets, assets).toUint128();

        // `assets` may be greater than `totalBorrowAssets` by 1.

        emit EventsLib.Repay(id, msg.sender, onBehalf, assets, shares);

            if (data.length > 0) IMorphoRepayCallback(msg.sender).onMorphoRepay(assets,
data);

              IERC20(marketParams.loanToken).safeTransferFrom(msg.sender,  address(this),
assets);

        return (assets, shares);

    }
```

So, if the attacker repays even `1 wei`, then the actual shares the user proxy holds are less than the shares that were assigned in the `LeveragePosition` struct. Ultimately, making the user not withdraw assets from the Morpho market.

Impact

Because an attacker can also repay `onbehalf` of any user, can repay the user's position to a small amount, ultimately leading the user to be unable to withdraw assets deposited.

Recommendation

Instead of repaying shares user position stores, only repay the number of shares the user proxy holds.

```
    function _repayAndWithdrawCollateralViaProxy(

        address userProxy,

        address collateralToken,

        address loanToken,

        uint256 amountLoan,

        uint256 amountCollateral,

        uint256 sharesBorrowed

    ) internal {

        MarketParams memory marketParams = s_marketParams[collateralToken][

            loanToken

        ];

+         uint256 sharesLeft = IERC20(share_token).balanceOf(userProxy); // not the exact
line about this but we can implement is similar way

+        sharesBorrowed = sharesLeft < sharesBorrowed? sharedLeft: sharesBorrowed

        _morphoRepay(userProxy, marketParams, amountLoan, sharesBorrowed);

        if (amountCollateral > 0) {

            _morphoWithdrawCollateralViaProxy(

                userProxy,

                marketParams,

                amountCollateral

            );

        }

    }
```

Status

Resolved

Medium

[M-01]SwapAggregator#_swapLoanTokenToCollateralToken, _swapCollateralTokenToLoanToken- Lack of slippage parameter will result in loss of user funds

Description

A big part of handling leverage and unleverage mechanisms is to swap the flash loan into a collateral token or a loan token, respectively. However, since slippage is hardcoded to 0, this could result in users being victims of sandwich attacks.

Vulnerability Details

The `_handleLeverage` and `_handleUleverage` given flashloan amount into either PT tokens or loan tokens via Pendle router involving and `swapExactTokenForPt` `swapExactPtForToken`. However, since the `minTokenOut` variable is hardcoded to zero, this results in no slippage protection for the swap transaction.

```
/**
  * @notice Swaps borrowed loan token to PT (collateral) via Pendle.
  * @param loanToken Token being swapped from (e.g., USDC).
  * @param collateralToken Token to be received (e.g., PT).
  * @param amountLoan Amount of loan token to swap.
  * @param approxParams Pendle slippage approximation settings.
  * @param pendleSwap Address used to route the swap.
  * @param swapData Swap path details.
  * @param limitOrderData Optional limit order info.
  * @return amountSwappedCollateralToken Amount of PT tokens received.
  */
 function _swapLoanTokenToCollateralToken(
     address loanToken,
```

```
        address collateralToken,
        uint256 amountLoan,
        ApproxParams memory approxParams,
        address pendleSwap,
        address tokenMintSy,
        SwapData memory swapData,
        LimitOrderData memory limitOrderData
    ) internal returns (uint256 amountSwappedCollateralToken) {
        _forceApprove(loanToken, address(i_pendleRouter), amountLoan);
        (amountSwappedCollateralToken, , ) = i_pendleRouter.swapExactTokenForPt(
            address(this),
            s_pendleMarket[collateralToken],
            0, //@audit lack of slippage
            approxParams,
            TokenInput({
                tokenIn: loanToken,
                netTokenIn: amountLoan,
                tokenMintSy: tokenMintSy,
                pendleSwap: pendleSwap,
                swapData: swapData
            }),
            limitOrderData
        );
    }
    /**
     * @notice Swaps PT (collateral) back to loan token via Pendle.
```

```
     * @param collateralToken Token to be swapped from (PT).
     * @param loanToken Token to be received (e.g., USDC).
     * @param amountCollateral Amount of PT to swap.
     * @param pendleSwap Address used to route the swap.
     * @param swapData Swap path details.
     * @param limitOrderData Optional limit order info.
     * @return amountSwappedLoanToken Amount of loan tokens received.
     */
    function _swapCollateralTokenToLoanToken(
        address collateralToken,
        address loanToken,
        uint256 amountCollateral,
        address pendleSwap,
        address tokenRedeemSy,
        SwapData memory swapData,
        LimitOrderData memory limitOrderData
    ) internal returns (uint256 amountSwappedLoanToken) {
        _safeApprove(
            collateralToken,
            address(i_pendleRouter),
            amountCollateral
        );
        (amountSwappedLoanToken, , ) = i_pendleRouter.swapExactPtForToken(
            address(this),
            s_pendleMarket[collateralToken],
            amountCollateral,
```

```
            TokenOutput({
                tokenOut: loanToken,
                minTokenOut: 0, //@audit lack of slippage
                tokenRedeemSy: tokenRedeemSy,
                pendleSwap: pendleSwap,
                swapData: swapData
            }),
            limitOrderData
        );
    }
```

As a result, in case of leverage, this could result in instant liquidation risk for the users where the resultant pt tokens after swap are very low and could possibly revert, leading to temporary `DOS`.

In case of unleverage, this could result in users having less or potentially no profit in return.

Impact

Loss of user funds

Recommendation

It is recommended to allow users to input a custom slippage parameter as per their needs.

Status

Resolved

[M-02] FlashLeverage#swapAndLeverage - User Can Leverage Less Than Actually Sent

Description

Because there is no proper validation on `FlashLeverage::swapAndLeverage` function, it allows users to deposit more tokens than actually needed, and these tokens are permanently locked within the contract.

Vulnerability Details

```
  function swapAndLeverage(
        address onBehalfOf,
>>>     SwapParams calldata swapParams,
>>>     LeverageParams calldata leverageParams
    )
        external
        validateOnBehalfOf(onBehalfOf)
        validateCollateralToken(leverageParams.collateralToken)
        validateAmount(swapParams.amountTokenIn)
        validateAmount(leverageParams.amountCollateral)
        validateDesiredLtv(
            leverageParams.desiredLtv,
            leverageParams.collateralToken,
            leverageParams.loanToken
        )
    {
        // Transfer tokens from user
        IERC20(swapParams.tokenIn).transferFrom(
            msg.sender,
```

```
            address(this),
>>>         swapParams.amountTokenIn
        );
        // Swap if needed
        if (swapParams.tokenIn != leverageParams.collateralToken) {
            _handleTokenSwap(leverageParams.collateralToken, swapParams);
        }
        // Call internal leverage
>>>     _leverage(onBehalfOf, leverageParams);
    }
```

As you can see in the above function, `SwapParams` & `LeverageParams` are completely different, and there is no validation that swapped tokens are equal to `leverageParams.amountCollateral`. Because of this, the user can provide more tokens and can only leverage fewer tokens, which end up locked within the `FlashLeverage` contract. Though it was the user's fault here, there is a very high chance it can happen frequently for dusted amounts, which end up aggregating to larger amounts.

Impact

Lack of proper validation, the user loses collateral tokens when they try to deposit more tokens than they want to leverage. Because of this, a malicious attacker can steal the tokens that were stuck. Please refer to [M-02] for detailed review.

Recommendation

Implement a scenario where the user can receive back the excess collateral token, and only the desired number of tokens gets leveraged.

Status

Resolved

[M-03] FlashLeverage#swapAndLeverage - Attacker can Leverage More Tokens than Actually Sent

Description

If, by chance, any collateral tokens are stuck within the `FlashLeverage` contract, then the attacker can leverage all the collateral tokens within the contract under his own account.

Vulnerability Details

As you can see in the function below, there is no proper validation on how many tokens are actually swapped with `leverageParams.amountCollateral`. The attacker can maliciously pass this `_handleTokenSwap` internal function and leverage his position with collateral held within the contract.

```
  function swapAndLeverage(
        address onBehalfOf,
>>>     SwapParams calldata swapParams,
>>>     LeverageParams calldata leverageParams
    )
        external
        validateOnBehalfOf(onBehalfOf)
        validateCollateralToken(leverageParams.collateralToken)
        validateAmount(swapParams.amountTokenIn)
        validateAmount(leverageParams.amountCollateral)
        validateDesiredLtv(
            leverageParams.desiredLtv,
            leverageParams.collateralToken,
            leverageParams.loanToken
        )
    {
```

```
        // Transfer tokens from user
        IERC20(swapParams.tokenIn).transferFrom(
            msg.sender,
            address(this),
            swapParams.amountTokenIn
        );
        // Swap if needed
        if (swapParams.tokenIn != leverageParams.collateralToken) {
>>>         _handleTokenSwap(leverageParams.collateralToken, swapParams);
        }
        // Call internal leverage
>>>     _leverage(onBehalfOf, leverageParams);
    }
```

Impact

Because of this, locked collateral tokens are stolen from a malicious user (attacker), and can be leveraged under a user proxy account.

Recommendation

Implement a scenario where the user can receive back the excess collateral token, and only the desired number of tokens gets leveraged. And validate such that there is no difference between tokens deposited and leveraged.

Status

Resolved

Low

[L-01] FlashLeverage#swapAndLeverage - Consider using safeTransferFrom instead.

Description

Some tokens that do not comply with `ERC20` like `USDT` return a bool instead of reverting.

Vulnerability Details

The `swapAndLeverage` function uses `transferFrom`. Since the protocol revolved around stablecoin yields, transferFrom will not be able to handle tokens that do not comply with `ERC20` standard, like `USDT`.

```
  /**
    * @notice Entry point for users. Swaps tokens if needed, approves, then leverages.
    * @param onBehalfOf Address to open the position on behalf of
    * @param swapParams Parameters including tokenIn, amountTokenIn
    * @param leverageParams Parameters for the leverage call
    */
   function swapAndLeverage(
       address onBehalfOf,
       SwapParams calldata swapParams,
       LeverageParams calldata leverageParams
   )
       external
       validateOnBehalfOf(onBehalfOf)
       validateCollateralToken(leverageParams.collateralToken)
       validateAmount(swapParams.amountTokenIn)
       validateAmount(leverageParams.amountCollateral)
```

```
        validateDesiredLtv(
            leverageParams.desiredLtv,
            leverageParams.collateralToken,
            leverageParams.loanToken )
    {
        // Transfer tokens from user
        IERC20(swapParams.tokenIn).transferFrom( //@audit consider using _transferIn here
            msg.sender,
            address(this),
            swapParams.amountTokenIn );
        // Swap if needed
        if (swapParams.tokenIn != leverageParams.collateralToken) {
            _handleTokenSwap(leverageParams.collateralToken, swapParams);
        }
        // Call internal leverage
        _leverage(onBehalfOf, leverageParams);
    }
```

Impact

Silent false return.

Recommendation

Even though the transaction will eventually revert in swap, it is still recommended to use `safeTransferFrom` instead.

Status

Resolved

[L-02] Contracts - Contract lacks a method to remove collateral support

Description

Even though the admin can configure the token configuration, there is no way to handle a scenario where the protocol needs to remove token support.

Recommendation

Consider adding a method to remove token support for emergency purposes.

Status

Acknowledged

[L-03] FlashLeverage & FlashLeverageCore - Lack of Recovery Function Lead to Stuck Accidental Transfers

Description

As there is no recover function in `FlashLeverageCore` & `FlashLeverage`, if any accidental transfers occur, then those tokens are permanently locked within the contract. There is no way a user can retrieve those tokens if those are not supported collateral tokens, but if they were admin, they can leverage and unleverage immediately to retrieve them back.

Impact

Accidental `ERC20` tokens get stuck permanently within the contract, causing the user to lose their tokens, as there is no way to retrieve them.

Recommendation

Implement a recover function as follows, and only adding it in `FlashLeverage` is enough, as the Core contract directly sends the asset to either Morpho Market or `FlashLeverage`, so no need to add it. But for a safe case, it is better to add it also in the core contract.

```
function recover(address token) external onlyOwner {

    uint256 balance = IERC20(token).balanceOf(address(this));

    _transferOut(token, msg.sender, balance);

}
```

Status

Resolved

[L-04] FlashLeverage - No Setter Function on YIELD_FEE

Description

No setter function for `YIELD_FEE` on `FlashLeverage` contract, because it was hardcoded to 10%. If the admin wants to change the yield fee, there is no function to change it, as there is no setter function.

Impact

Because there is no setter function for `YIELD_FEE`, users may feel a 10% heavy amount is incurred on them. If the protocol wants to attract users, the only option they have is by reducing `YIELD_FEE`, but here it isn't possible.

Recommendation

Implement a setter function with a maximum of 10%

Status

Resolved

[L-05] FlashLeverage & FlashLeverageCore - No Zero Address Validation

Description

There are many instances across contracts where zero validation was not properly addressed. Here are a few functions:

1. `FlashLeverage::addSupportedCollateralTokens` - on Pendle market address, and would recommend implementing this function in the same way as `FlashLeverageCore` contract, i.e., by checking the provided Pendle market PT with collateral token
2. `FlashLeverageCore & FlashLeverage` - on constructor inputs

Recommendation

Add the zero address checks

Status

Resolved

[L-06] FlashLeverageCore#_revertIfEffectiveLtvTooHigh - Consumes More Gas

Description

`FlashLeverage::_revertIfEffectiveLtvTooHigh` function is being called way later to verify the state to proceed user position on Morhpo market, which consumes lot of gas because before it is called, flashloan and Pendle swap happen, which interacts with the external protocol, which consumes a lot of gas.

Though in most cases the call would be successful, there is a certain probability that LTV becomes higher than expected, which ends up consuming more gas.

Recommendation

Not sure about the decimal calculation, but the idea was the same, because of this, we can avoid a lot of gas

```
function leverage(
        address onBehalfOf,
        LeverageParams calldata params
    )
        external
        onlyManager
        validateCollateralToken(params.collateralToken, params.loanToken)
        validateAmountCollateral(params.amountCollateral)
        validateDesiredLtv(
            params.desiredLtv,
            params.collateralToken,
            params.loanToken
        )
    {
```

```
        address collateralToken = params.collateralToken;
        address loanToken = params.loanToken;
        uint256 amountCollateral = params.amountCollateral;
        _transferIn(collateralToken, msg.sender, amountCollateral);
        // FlashLoan Related
        uint256 amountFlashLoan = calcLeverageFlashLoan(
            params.desiredLtv,
            collateralToken,
            loanToken,
            amountCollateral
        );
+        uint256 collateralValue = getCollateralValueInLoanToken(
+            collateralToken,
+            loanToken,
+            amountCollateral
+        );
+        // Total position value = collateralValue / (1 - LTV)
+        totalPositionValue = collateralValue.divDown(
+            Math.ONE - desiredLtv
+        );
+        _revertIfEffectiveLtvTooHigh(
+            params.desiredLtv,
+            collateralToken,
+            loanToken,
+            totalPositionValue,
+            amountFlashLoan
```

```
+         );
          bytes memory data = abi.encode(
              Action.LEVERAGE,
              onBehalfOf, // user
              params.desiredLtv,
              collateralToken,
              loanToken,
              amountCollateral,
              params.approxParams,
              params.pendleSwap,
              params.tokenMintSy,
              params.swapData,
              params.limitOrderData
          );
          i_morpho.flashLoan(loanToken, amountFlashLoan, data);
      }
```

Status

Acknowledged

[L-07] FlashLeverage#_unleverage - Using Deposited Amount From Leverage Opened Instant May Lead to Differ in Slashing Yield Fee

Description

The slashing yield fee wasn't properly calculated in `FlashLeverage::_unleverage` function, because while getting the `totalAmountDeposited` amount, it directly retrieves from the leverage position, but if there is any peg difference, the value may be greater or less than the actual, resulting in a slash less or more yield fee, respectively.

Vulnerability Details

As you can see in the function below, `totalAmountDeposited` is getting from the `LeveragePosition` struct, which gets from the time the user opened the leveraged position. So, if there is a heavy shift in peg price, then `totalAmountDeposited`, actually received tokens, as they were obtained based on the oracle price.

```
    function _unleverage(
        address user,
        uint256 positionId,
        LeveragePosition storage position,
        address pendleSwap,
        address tokenRedeemSy,
        SwapData calldata swapData,
        LimitOrderData calldata limitOrderData
    ) internal returns (uint256 amountReturned) {
        ...
>>>     uint256 totalAmountDeposited = position
            .amountCollateralInLoanToken
            .scaleTo(loanTokenDecimals, Math.STANDARD_DECIMALS);
        uint256 totalAmountReceived = _selfBalance(position.loanToken).scaleTo(
            loanTokenDecimals,
```

```
            Math.STANDARD_DECIMALS
        );
        // Handle yield fee calculation and transfer
        uint256 amountFee;
        if (totalAmountReceived > totalAmountDeposited) {
>>>         uint256 yieldGenerated = totalAmountReceived - totalAmountDeposited;
>>>         amountFee = yieldGenerated.mulDown(YIELD_FEE);
            _transferOut(
                position.loanToken,
                s_treasury,
                amountFee.scaleTo(Math.STANDARD_DECIMALS, loanTokenDecimals)
            );
        }
        // Transfer remaining amount to user
>>>     amountReturned = (totalAmountReceived - amountFee).scaleTo(
            Math.STANDARD_DECIMALS,
            loanTokenDecimals
        );
        _transferOut(position.loanToken, user, amountReturned);
        emit LeveragePositionClosed(user, positionId, amountReturned);
    }
```

Because of this discrepancy, users can receive more tokens than intended or lose more tokens than intended based on the actual tokens received from the Morpho market. Even though retrieving the deposited asset from Oracle was gas-consuming, it was a recommended solution for worst-case scenarios.

Impact

Impact wasn't that severe because it only lost a tiny amount, as stable coins mostly stay within the $1 range with ±$0.01. But if there arises any depeg, then in the worst-case scenario, it goes beyond $0.1, which makes user assets get slashed as uncertain.

Recommendation

Try to get the `totalAmountDeposited` value from the oracle using the function `FlashLeverageCore::getCollateralValueInLoanToken`

Status

Acknowledged

QA

[Q-01] Consider implementing an emergency pause mechanism

Description

The contract does not have any mechanism to pause them in case of an emergency.

Recommendation

It is recommended to inherit OpenZeppelin's Pausable.sol and consider using the `whenNotPaused` modifier over critical functions.

Status

Resolved

Centralisation

The SpiralStake project values decentralisation alongside security and utility.

The protocol's functions are designed to operate independently, minimising reliance on the internal team and hardcoded values, while maintaining robust security and functionality.

Conclusion

After Hashlock's analysis, the Spiral Stake project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved or acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au